

THUẬT TOÁN TÌM KIẾM CÓ THÔNG TIN (INFORMED SEARCH ALGORITHMS)

CHƯƠNG 4, PHẦN 1–2

Phác thảo

- ◇ Tìm kiếm đầu tiên tốt nhất
- ◇ tìm kiếm A^*
- ◇ Chẩn đoán

Đánh giá: Tìm kiếm cây

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
loop do
  if fringe is empty then return failure
  node ← REMOVE-FRONT(fringe)
  if GOAL-TEST[problem] applied to STATE(node) succeeds return node
  fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

Chiến lược được xác định bằng cách chọn thứ tự **mở rộng nút**

Tìm kiếm đầu tiên tốt nhất

Idea: sử dụng hàm đánh giá cho mỗi nút

– ước tính về "sự mong muốn"

⇒ Mở rộng nút chưa được mở rộng mong muốn nhất

Triển khai:

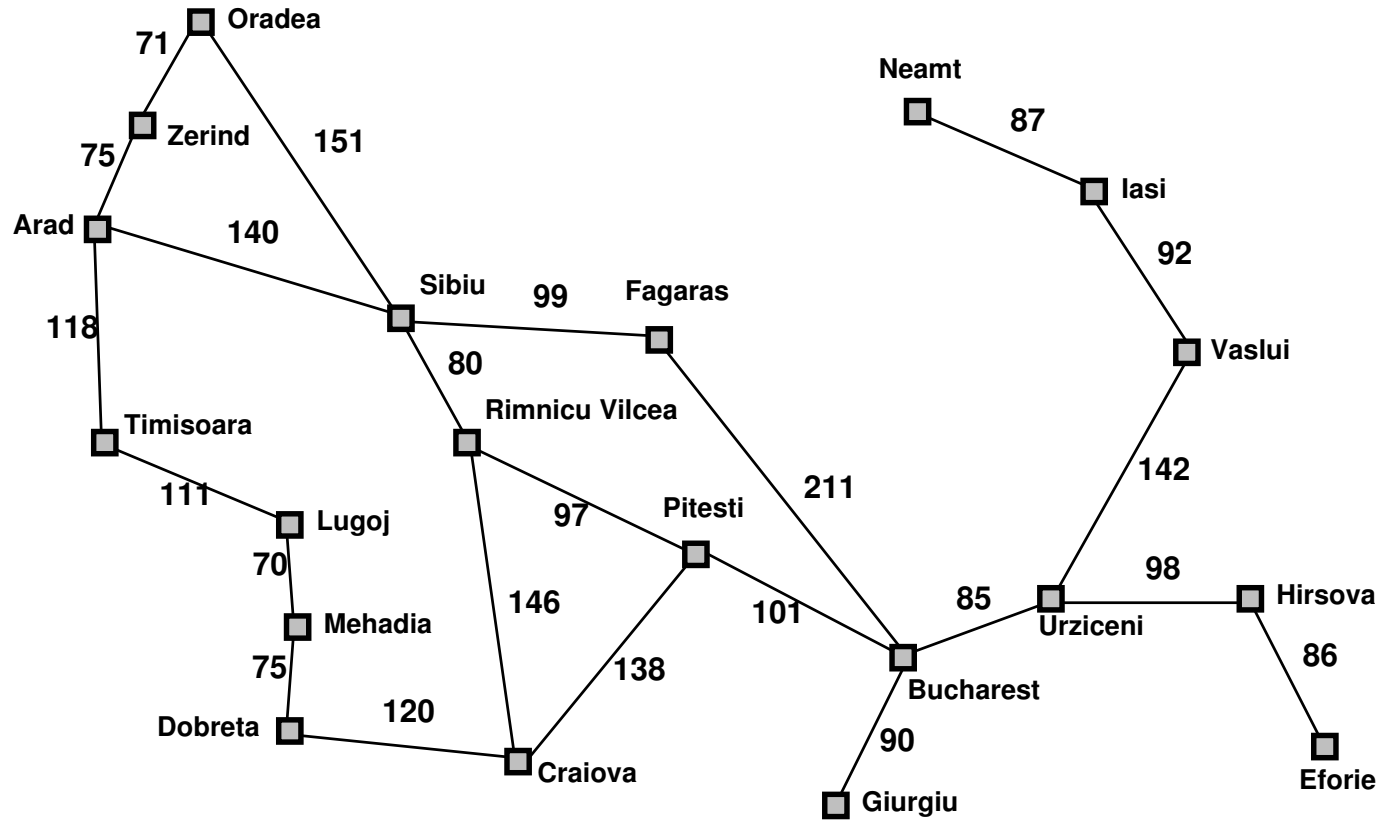
fringe là hàng đợi được sắp xếp theo thứ tự mong muốn giảm dần

Các trường hợp đặc biệt:

tìm kiếm tham lam

tìm kiếm A^*

Romania với chi phí bước tính bằng km



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Tham lam tìm kiếm

Hàm đánh giá $h(n)$ (**h**euristic)

= ước tính chi phí từ n đến mục tiêu gần nhất

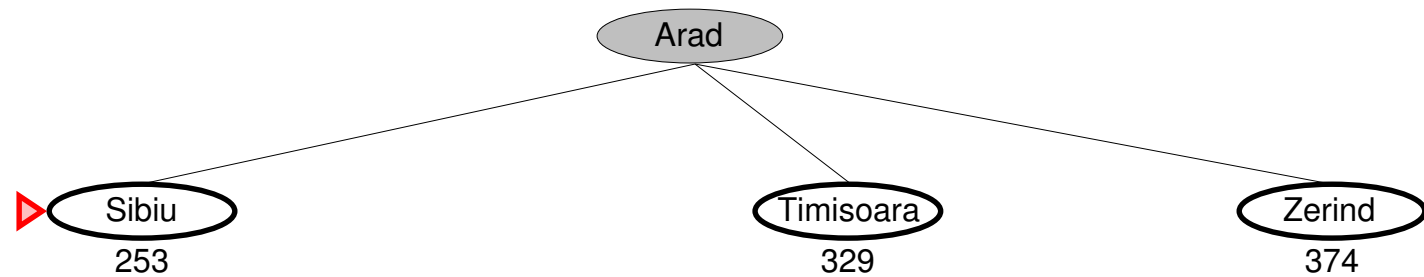
Ví dụ: $h_{SLD}(n)$ = khoảng cách đường thẳng từ n đến Bucharest

Tìm kiếm tham lam mở rộng nút **xuất hiện** gần mục tiêu nhất

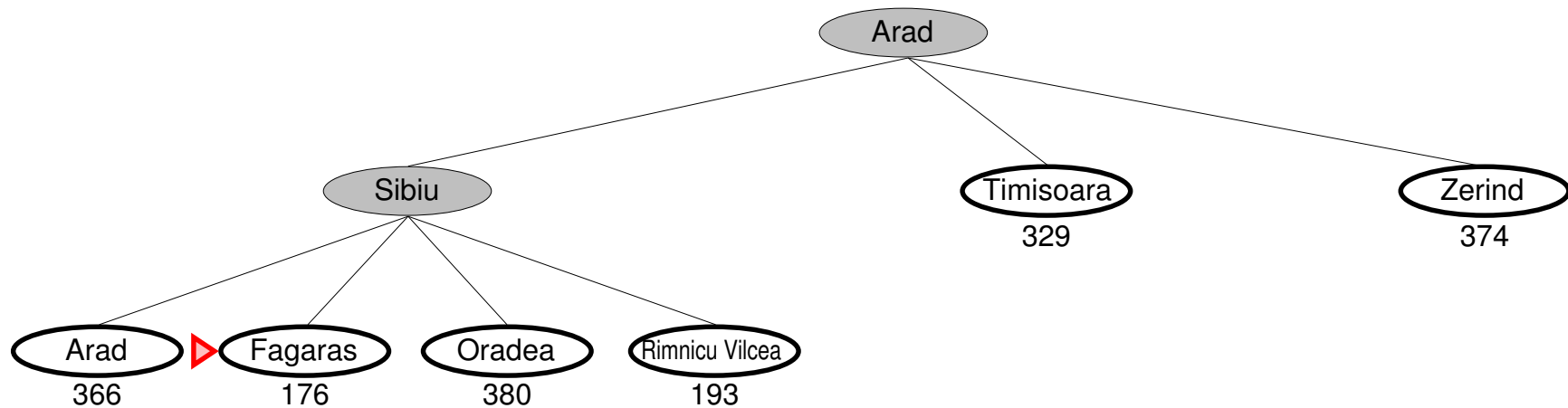
Ví dụ về tìm kiếm tham lam

▶ Arad
366

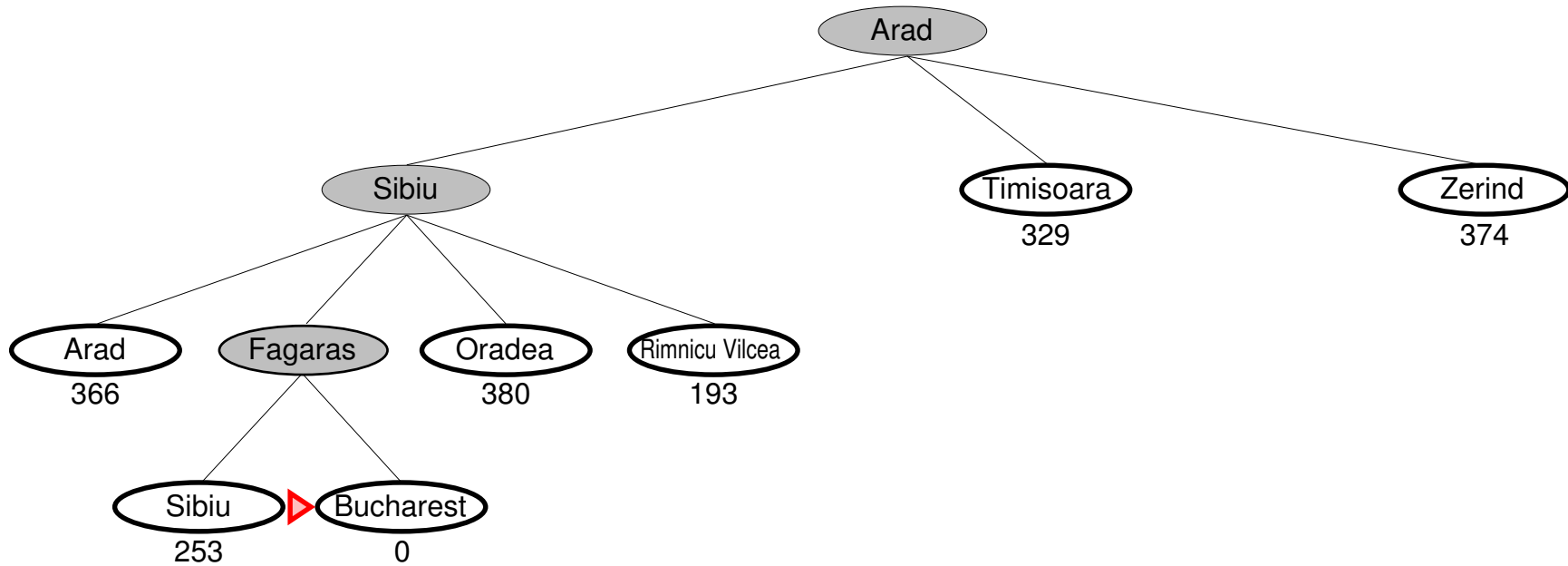
Ví dụ về tìm kiếm tham lam



Ví dụ về tìm kiếm tham lam



Ví dụ về tìm kiếm tham lam



Thuộc tính tìm kiếm tham lam

Hoàn thành??

Thuộc tính tìm kiếm tham lam

Hoàn thành?? Không-có thể bị mắc kẹt trong các vòng lặp, ví dụ: với Oradea làm mục tiêu,

Iasi → Neamt → Iasi → Neamt →

Hoàn thành trong không gian hữu hạn với việc kiểm tra trạng thái lặp lại

Thời gian??

Thuộc tính tìm kiếm tham lam

Hoàn thành?? Không-có thể bị mắc kẹt trong các vòng lặp, ví dụ:

lasi $\rightarrow$ Neamt $\rightarrow$ lasi $\rightarrow$ Neamt $\rightarrow$

Hoàn thành trong không gian hữu hạn với việc kiểm tra trạng thái lặp lại

Thời gian?? $O(b^m)$, nhưng phương pháp phỏng đoán tốt có thể mang lại sự cải thiện đáng kể

Không gian??

Thuộc tính tìm kiếm tham lam

Hoàn thành?? Không—có thể bị mắc kẹt trong các vòng lặp, ví dụ:

lasi → Neamt → lasi → Neamt →

Hoàn thành trong không gian hữu hạn với việc kiểm tra trạng thái lặp lại

Thời gian?? $O(b^m)$, nhưng phương pháp phỏng đoán tốt có thể mang lại sự cải thiện đáng kể

Space?? $O(b^m)$ —giữ tất cả các nút trong bộ nhớ

Tối ưu??

Thuộc tính tìm kiếm tham lam

Hoàn thành?? Không—có thể bị mắc kẹt trong các vòng lặp, ví dụ:

lasi → Neamt → lasi → Neamt →

Hoàn thành trong không gian hữu hạn với việc kiểm tra trạng thái lặp lại

Thời gian?? $O(b^m)$, nhưng phương pháp phỏng đoán tốt có thể mang lại sự cải thiện đáng kể

Space?? $O(b^m)$ —giữ tất cả các nút trong bộ nhớ

Tối ưu?? Không

A* tìm kiếm

Ý tưởng: tránh mở rộng những con đường vốn đã tốn kém

Hàm đánh giá $f(n) = g(n) + h(n)$

$g(n)$ = chi phí cho đến nay để đạt được n

$h(n)$ = chi phí ước tính cho mục tiêu từ n

$f(n)$ = tổng chi phí ước tính của đường đi qua n tới mục tiêu

Tìm kiếm A* sử dụng phương pháp phỏng đoán **admissible**

tức là, $h(n) \leq h^*(n)$ trong đó $h^*(n)$ là chi phí **true** từ n .

(Cũng yêu cầu $h(n) \geq 0$, vì vậy $h(G) = 0$ cho bất kỳ mục tiêu nào G .)

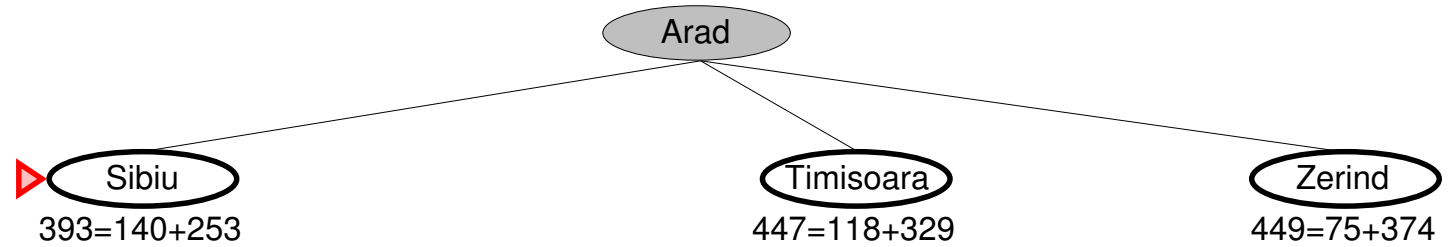
Ví dụ: $h_{\text{SLD}}(n)$ không bao giờ đánh giá quá cao khoảng cách đường thực tế

Định lý: tìm kiếm A* là tối ưu

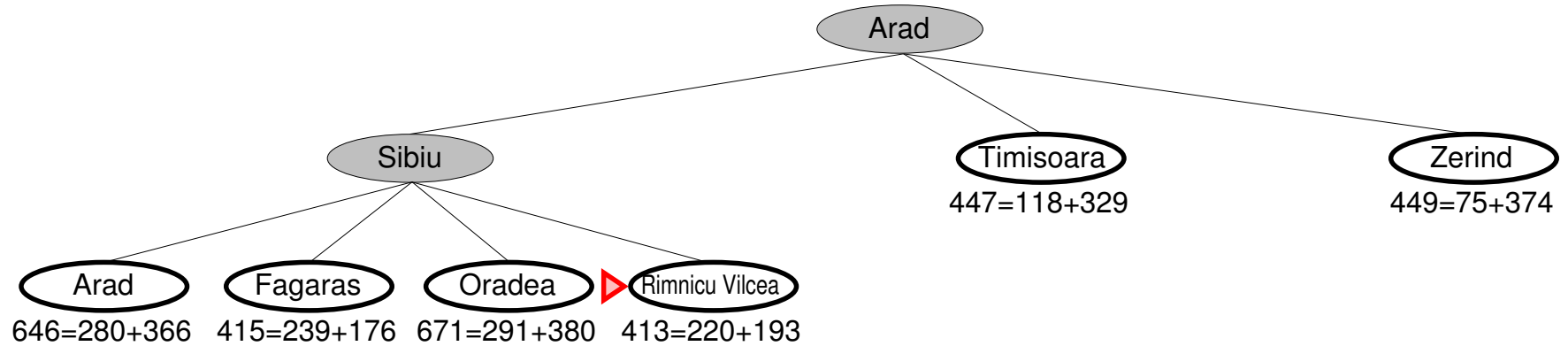
A^* ví dụ tìm kiếm

▶ Arad
 $366=0+366$

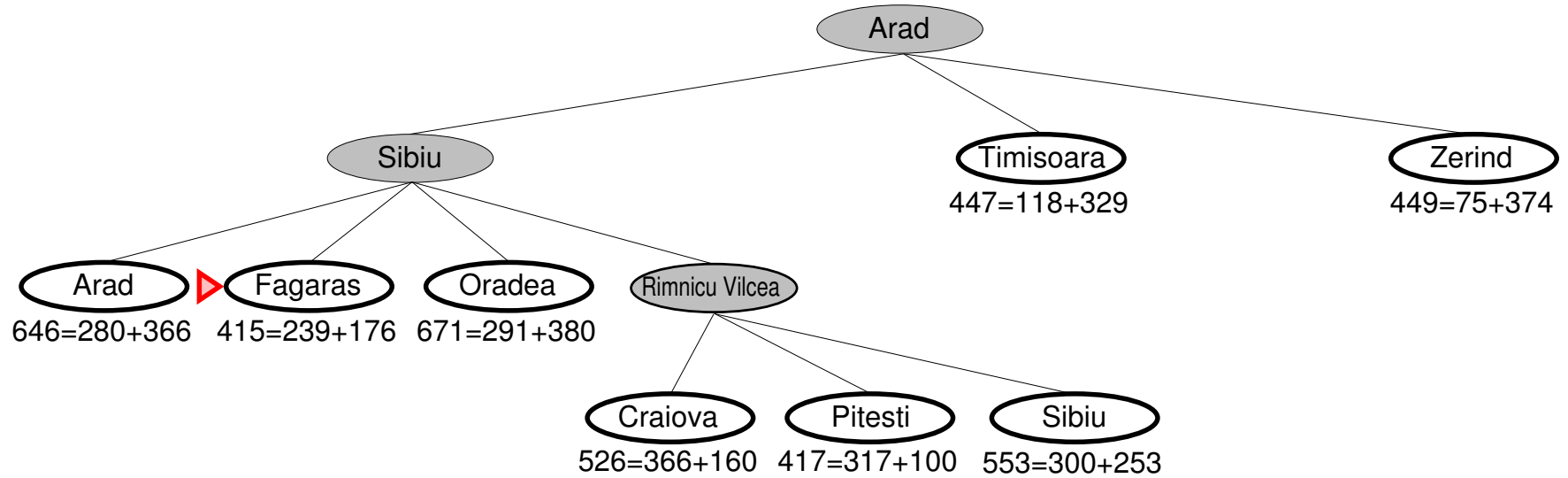
A^* ví dụ tìm kiếm



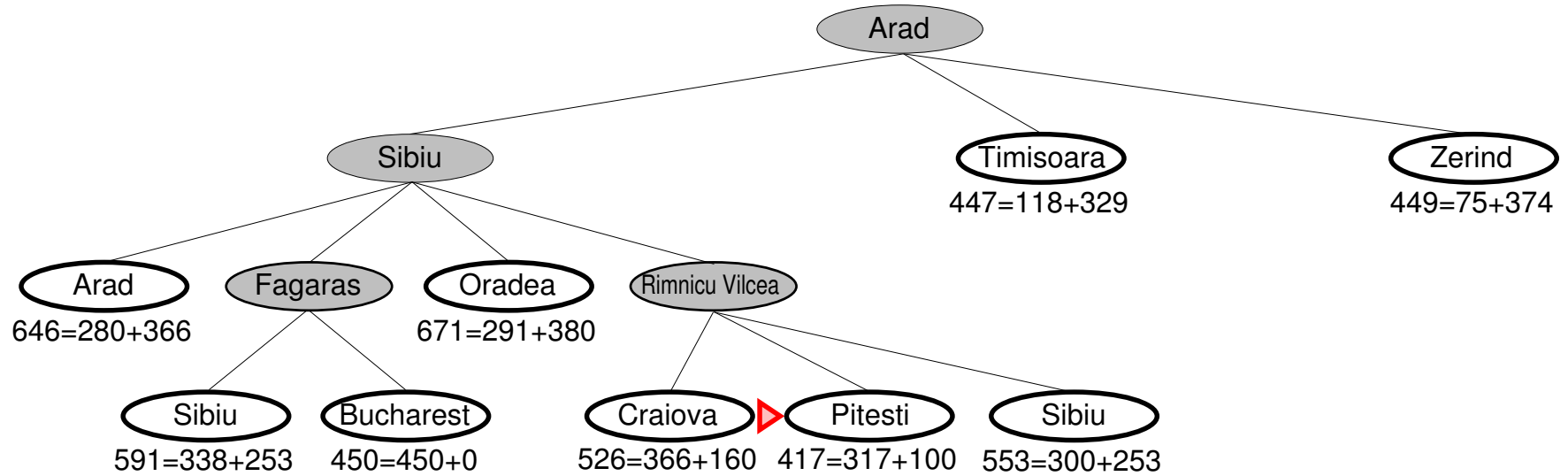
A* ví dụ tìm kiếm



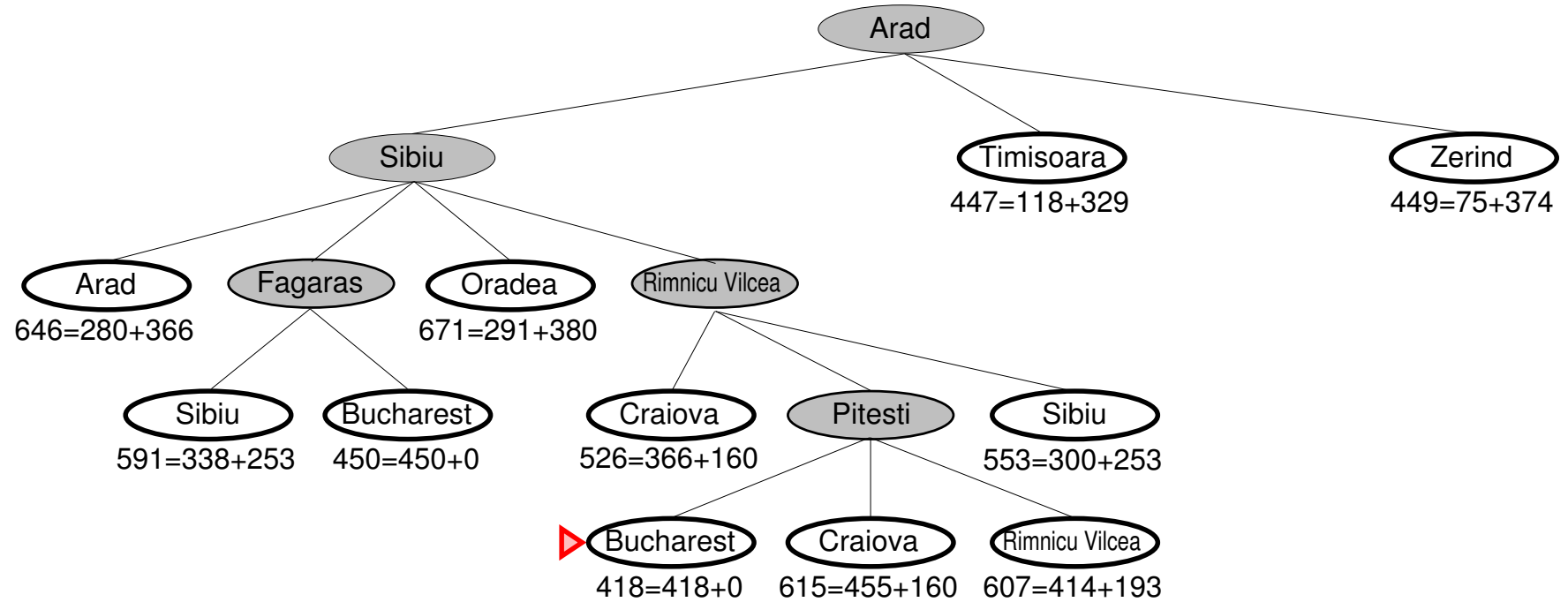
A* ví dụ tìm kiếm



A* ví dụ tìm kiếm

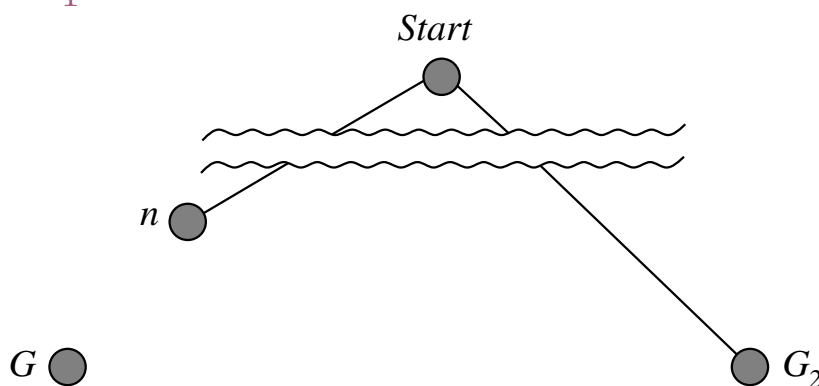


A* ví dụ tìm kiếm



Tính tối ưu của A^* (bằng chứng tiêu chuẩn)

Giả sử một số mục tiêu dưới mức tối ưu G_2 đã được tạo và đang trong hàng đợi. Đặt n là một nút chưa được mở rộng trên con đường ngắn nhất tới mục tiêu tối ưu G_1 .



$$\begin{aligned} f(G_2) &= g(G_2) && \text{since } h(G_2) = 0 \\ &> g(G_1) && \text{since } G_2 \text{ is suboptimal} \\ &\geq f(n) && \text{since } h \text{ is admissible} \end{aligned}$$

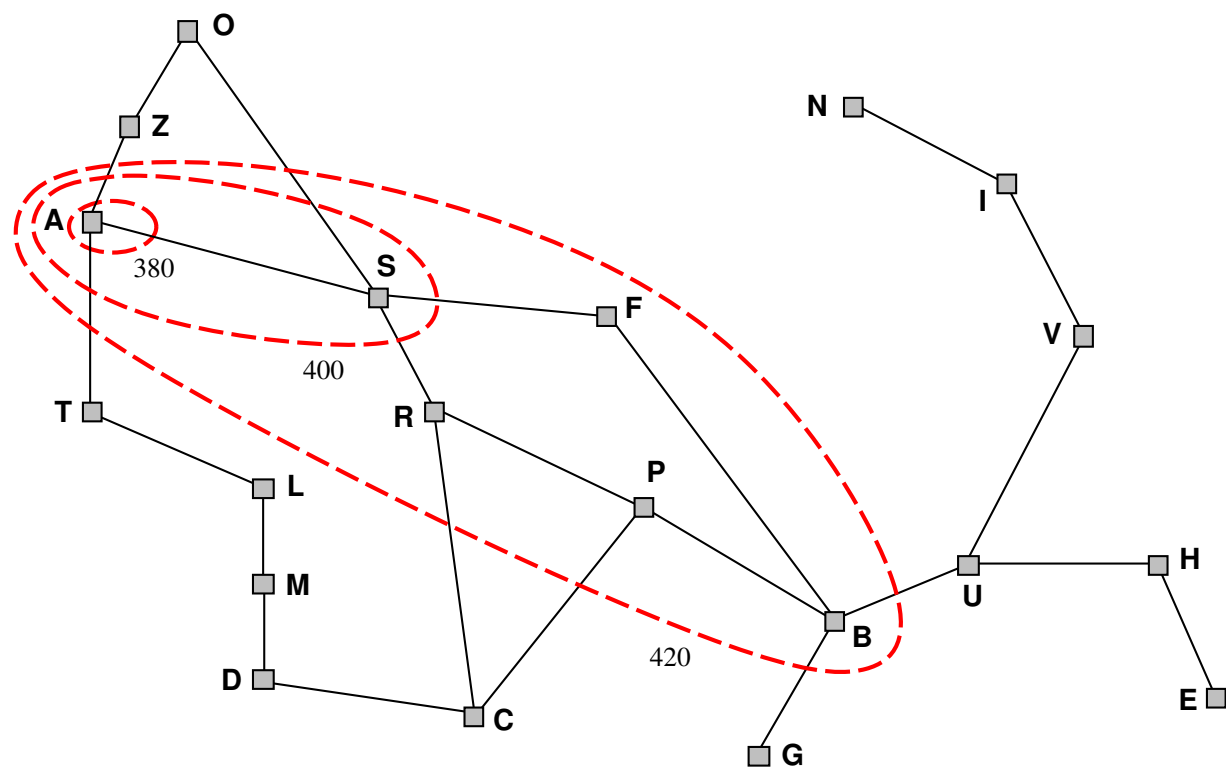
Vì $f(G_2) > f(n)$, A^* sẽ không bao giờ chọn G_2 để mở rộng

Tính tối ưu của A^* (hữu ích hơn)

Bổ đề: A^* mở rộng các nút theo thứ tự tăng dần giá trị f^*

Dần dần thêm “ f -contours” của các nút (xem thêm các lớp theo chiều rộng đầu tiên)

Đường viền i có tất cả các nút có $f = f_i$, trong đó $f_i < f_{i+1}$



Thuộc tính của A^*

Hoàn thành??

Thuộc tính của A^*

Hoàn thành?? Có, trừ khi có vô số nút có $f \leq f(G)$

Thời gian??

Thuộc tính của A^*

Hoàn thành?? Có, trừ khi có vô số nút có $f \leq f(G)$

Thời gian?? Hàm mũ theo [lỗi tương đối trong $h \times$ độ dài của soln.]

Không gian??

Thuộc tính của A^*

Hoàn thành?? Có, trừ khi có vô số nút có $f \leq f(G)$

Thời gian?? Hàm mũ theo [lỗi tương đối trong $h \times$ độ dài của soln.]

Space?? Giữ tất cả các nút trong bộ nhớ

Tối ưu??

Thuộc tính của A^*

Hoàn thành?? Có, trừ khi có vô số nút có $f \leq f(G)$

Thời gian?? Hàm mũ theo [lỗi tương đối trong $h \times$ độ dài của soln.]

Space?? Giữ tất cả các nút trong bộ nhớ

Tối ưu?? Có—không thể mở rộng f_{i+1} cho đến khi f_i kết thúc

A^* mở rộng tất cả các nút bằng $f(n) < C^*$

A^* mở rộng một số nút bằng $f(n) = C^*$

A^* không mở rộng nút nào với $f(n) > C^*$

Chứng minh bổ đề: Tính nhất quán

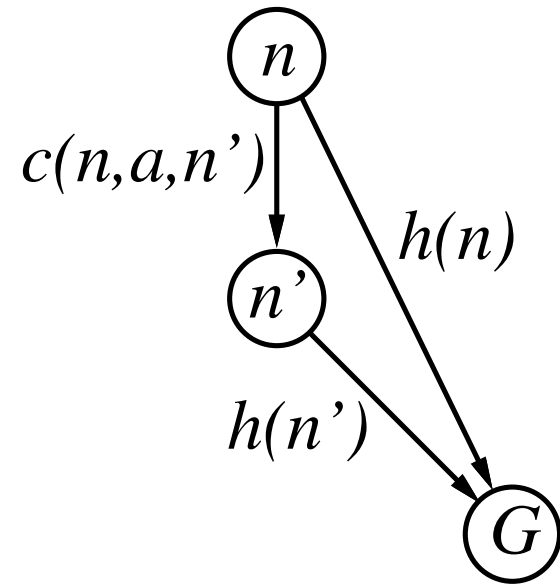
Một phương pháp phỏng đoán là **nhất quán** nếu

$$h(n) \leq c(n, a, n') + h(n')$$

Nếu h nhất quán, chúng ta có

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

Tức là, $f(n)$ không giảm dọc theo bất kỳ đường dẫn nào.



Các phương pháp phỏng đoán được chấp nhận

Ví dụ: đối với câu đố 8:

$h_1(n)$ = số ô bị đặt sai vị trí

$h_2(n)$ = tổng khoảng cách **Manhattan**

(tức là số ô vuông từ vị trí mong muốn của mỗi ô)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$$h_2(S) = ??$$

$$h_2(S) = ??$$

Các phương pháp phỏng đoán được chấp nhận

Ví dụ: đối với câu đố 8:

$h_1(n)$ = số ô bị đặt sai vị trí

$h_2(n)$ = tổng khoảng cách [Manhattan](#)

(tức là số ô vuông từ vị trí mong muốn của mỗi ô)

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$h_2(S) = ??$ 6

$h_2(S) = ??$ $4+0+3+3+1+0+2+1 = 14$

Sự thống trị

Nếu $h_2(n) \geq h_1(n)$ cho tất cả n (cả hai đều được chấp nhận) thì h_2 chiếm ưu thế h_1 và tốt hơn cho tìm kiếm

Chi phí tìm kiếm điển hình:

$d = 14$ IDS = 3.473.941 nút

$A^*(h_1) = 539$ nút

$A^*(h_2) = 113$ nút

$d = 24$ IDS $\approx$ 54.000.000.000 nút

$A^*(h_1) = 39.135$ nút

$A^*(h_2) = 1.641$ nút

Với bất kỳ phương pháp phỏng đoán được chấp nhận nào h_a, h_b ,

$$h(n) = \max(h_a(n), h_b(n))$$

cũng được chấp nhận và chiếm ưu thế h_a, h_b

Các vấn đề đã được giải quyết

Các phương pháp phỏng đoán được chấp nhận có thể được rút ra từ **chính xác**

chi phí giải pháp của phiên bản **thoải mái** của vấn đề

Nếu các quy tắc của câu đố 8 được nói lỏng để một viên gạch có thể di chuyển **bất cứ nơi nào**, sau đó $h_1(n)$ đưa ra giải pháp ngắn nhất

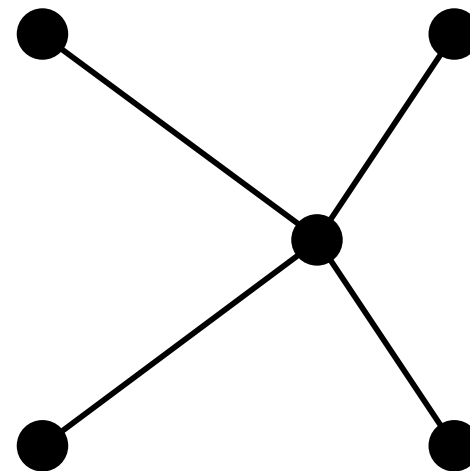
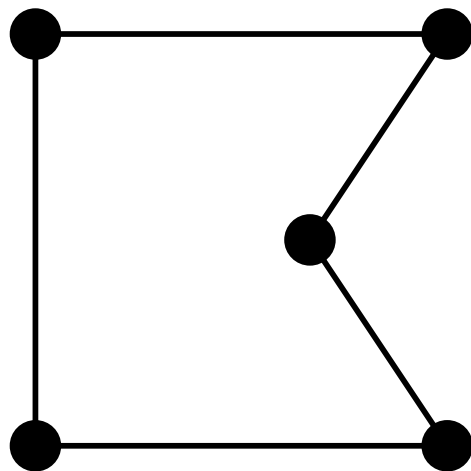
Nếu các quy tắc được nói lỏng để một ô có thể di chuyển tới **bất kỳ ô liền kề nào Square**, sau đó $h_2(n)$ cho lời giải ngắn nhất

Điểm mấu chốt: chi phí giải pháp tối ưu của một vấn đề thoải mái không lớn hơn chi phí giải pháp tối ưu của bài toán thực

Các vấn đề đã được giải quyết tiếp.

Ví dụ nổi tiếng: **vấn đề của nhân viên bán hàng du lịch** (TSP)

Tìm chuyến đi ngắn nhất đến thăm tất cả các thành phố đúng một lần



Cây bao trùm tối thiểu có thể được tính trong $O(n^2)$
và là giới hạn dưới của hành trình (mở) ngắn nhất

Tóm tắt

Hàm heuristic ước tính chi phí của các đường đi ngắn nhất

Phương pháp phỏng đoán tốt có thể giảm đáng kể chi phí tìm kiếm

Tìm kiếm đầu tiên tốt nhất tham lam mở rộng thấp nhất h

– không đầy đủ và không phải lúc nào cũng tối ưu

Tìm kiếm A^* mở rộng ở mức thấp nhất $g + h$

– đầy đủ và tối ưu

– cũng có hiệu quả tối ưu (tối đa các điểm ngắt, để tìm kiếm chuyển tiếp)

Các phương pháp phỏng đoán có thể chấp nhận được có thể được rút ra từ lời giải chính xác của các bài toán thoả mái